

Catenary curve

Let us define a catenary curve as a plane flexible curve with uniform linear density and uniform cross-section, freely suspended by its fixed ends in a gravitational field. The length of the curve is prescribed as a parameter, and it cannot be smaller than the straight-line distance between the fixed endpoints. It is assumed that the cross-section of the curve is much smaller than its length and therefore does not affect its elastic properties. Our task is to determine the shape of the curve for a given length and given endpoint positions (x_1, y_1) and (x_2, y_2) , which may lie at different heights ($y_1 \neq y_2$), as well as the tensile stress developed in the curve — the most important parameter from the point of view of applications.

There are two approaches to this problem. From the mechanical point of view, one should consider the tensile stress that arises in the curve, directed tangentially to its equilibrium shape, together with its vertical and horizontal projections. Equilibrium of the curve means that each infinitesimal element is subject to a set of forces, including gravity, which balance one another. From the geometrical point of view, the equilibrium curve is formed according to the principle of minimization of its potential energy in the gravitational field: each infinitesimal element tends to move as low as possible, while the total length of the curve remains constrained. In this approach, the mechanics of forces is not visible explicitly, although it is the microscopic cause of the equilibrium shape.

The potential energy of the curve $y(x)$ in the gravitational field is:

$$J(y) = \rho g \int_{x_1}^{x_2} y(x) \sqrt{1 + \dot{y}^2(x)} dx \quad (1)$$

where $\dot{y} = dy/dx$, ρ is the linear mass density of the curve (kg/m), and $g \approx 9.8 \text{ m/s}^2$ is the acceleration due to gravity. At the same time, it is necessary to impose the constraint on the length of the curve:

$$L = \int_{x_1}^{x_2} \sqrt{1 + \dot{y}^2(x)} dx \quad (2)$$

Thus, the optimization problem can be written in terms of the following functionals:

$$\begin{cases} J(y) = \rho g \int_{x_1}^{x_2} y(x) \sqrt{1 + \dot{y}^2(x)} dx \rightarrow \min \\ L = \int_{x_1}^{x_2} \sqrt{1 + \dot{y}^2(x)} dx \end{cases} \quad (3)$$

Using the method of Lagrange multipliers, we write the functional that incorporates the constraint:

$$\begin{aligned} J(y) &= \rho g \int_{x_1}^{x_2} y(x) \sqrt{1 + \dot{y}^2(x)} dx + \lambda \int_{x_1}^{x_2} \sqrt{1 + \dot{y}^2(x)} dx = \\ &= \int_{x_1}^{x_2} (\rho g y(x) + \lambda) \sqrt{1 + \dot{y}^2(x)} dx \rightarrow \min \end{aligned} \quad (4)$$

where λ is a constant to be determined.

Let us write the Euler equation for $F(x, y, \dot{y}) = (\rho g y(x) + \lambda) \sqrt{1 + \dot{y}^2(x)}$ in (4):

$$F_y - \frac{d}{dx} F_{\dot{y}} = 0 \quad (5)$$

Calculating the derivatives in (5), we obtain:

$$(\rho g y + \lambda) \ddot{y} = \rho g (1 + \dot{y}^2) \quad (6)$$

To solve this differential equation, we introduce the substitution $p = \dot{y}$. Then, $\ddot{y} = \dot{p} = \dot{y} \frac{dp}{dy} = p \frac{dp}{dy}$.

Substituting this expression into (6) and separating the variables, we obtain:

$$\frac{p dp}{\rho g (1 + p^2)} = \frac{dy}{(\rho g y + \lambda)} \quad (7)$$

Integrating both sides of (7), we find the function $p = \dot{y} = v(y)$, which becomes a first-order differential equation for $y(x)$. Recovering y from this equation, we finally obtain the equation of the curve:

$$y(x) = a \cosh\left(\frac{x-b}{a}\right) - \frac{\lambda}{\rho g} = \frac{a}{2} \left[\exp\left(\frac{x-b}{a}\right) + \exp\left(-\frac{x-b}{a}\right) \right] - \frac{\lambda}{\rho g} \quad (8)$$

where a and b are arbitrary constants obtained after evaluating the indefinite integrals.

In our solution, there are three unknown constants, which are determined from the boundary conditions and the condition of fixed length:

$$y(x_1) = y_1 \rightarrow \frac{a}{2} \left[\exp\left(\frac{x_1-b}{a}\right) + \exp\left(-\frac{x_1-b}{a}\right) \right] - \frac{\lambda}{\rho g} = y_1 \quad (9)$$

$$y(x_2) = y_2 \rightarrow \frac{a}{2} \left[\exp\left(\frac{x_2-b}{a}\right) + \exp\left(-\frac{x_2-b}{a}\right) \right] - \frac{\lambda}{\rho g} = y_2 \quad (10)$$

$$\int_{x_1}^{x_2} \sqrt{1 + \dot{y}^2(x)} dx = L \rightarrow \frac{a}{2} \left[\exp\left(\frac{x_2-b}{a}\right) - \exp\left(-\frac{x_2-b}{a}\right) - \exp\left(\frac{x_1-b}{a}\right) + \exp\left(-\frac{x_1-b}{a}\right) \right] = L \quad (11)$$

Subtracting (10) from (9), we obtain another equation for a and b , while either (9) or (10) itself can be used to calculate λ after a and b have already been determined. Thus, to determine a and b , we obtain the following equations:

$$a \left[\exp\left(\frac{x_2-b}{a}\right) - \exp\left(-\frac{x_2-b}{a}\right) - \exp\left(\frac{x_1-b}{a}\right) + \exp\left(-\frac{x_1-b}{a}\right) \right] = 2L \quad (12)$$

$$a \left[\exp\left(\frac{x_1-b}{a}\right) + \exp\left(-\frac{x_1-b}{a}\right) - \exp\left(\frac{x_2-b}{a}\right) - \exp\left(-\frac{x_2-b}{a}\right) \right] = 2(y_1 - y_2) \quad (13)$$

Equations (12) and (13) must be solved numerically, but it is better first to reduce them to a single equation. Let us introduce new variables $q = (x_1 - b)/a$ and $r = (x_2 - b)/a$. Then (12) and (13) can be written as:

$$\frac{x_2 - x_1}{r - q} [\exp(r) - \exp(-r) - \exp(q) + \exp(-q)] = 2L \quad (14)$$

$$\frac{x_2 - x_1}{r - q} [\exp(q) + \exp(-q) - \exp(r) - \exp(-r)] = 2(y_1 - y_2) \quad (15)$$

Subtracting and adding (14) and (15), we obtain:

$$\frac{x_2 - x_1}{r - q} \exp(-r) [\exp(r - q) - 1] = L + (y_1 - y_2) \quad (16)$$

$$\frac{x_2 - x_1}{r - q} \exp(r) [1 - \exp(-(r - q))] = L - (y_1 - y_2) \quad (17)$$

Dividing (17) by (16), we express r through $(r - q)$:

$$r = \frac{1}{2} \ln \left[\exp(r - q) \times \frac{L - (y_1 - y_2)}{L + (y_1 - y_2)} \right] \quad (18)$$

For the expression under the logarithm in (18) to be positive, it is necessary to require $(y_1 - y_2) < L$. However, this condition is automatically satisfied if we take into account that the minimum length of the curve between the fixed endpoints cannot be smaller than $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Substituting r into (17), we obtain an equation with respect to $(r - q)$, which can be solved numerically:

$$\frac{\exp(r - q) - 1}{(r - q) \exp\left(\frac{r - q}{2}\right)} = \frac{\sqrt{L^2 - (y_1 - y_2)^2}}{x_2 - x_1} \quad (19)$$

We shall assume that $x_1 < x_2$ always holds. Then $(r - q)$ is also positive. The function on the left-hand side of (19), considered as a function of the variable $s = r - q$, is smooth and monotonic, with a removable singularity at $s = 0$. In order not to worry about the interval in which the root is sought, we shall solve equation (19) by Newton's method for the following function:

$$f(s) = \frac{\exp(s) - 1}{s \exp\left(\frac{s}{2}\right)} - \frac{\sqrt{L^2 - (y_1 - y_2)^2}}{x_2 - x_1} \quad (20)$$

Its derivative is:

$$\dot{f}(s) = \frac{\frac{s}{2} \exp\left(\frac{3s}{2}\right) - \exp\left(\frac{3s}{2}\right) + \exp\left(\frac{s}{2}\right) + \frac{s}{2} \exp\left(\frac{s}{2}\right)}{s^2 \exp(s)} \quad (21)$$

[The iterative Newton method](#) is constructed according to the following scheme:

$$s_{n+1} = s_n - \frac{f(s_n)}{\dot{f}(s_n)} \quad (22)$$

As an initial point, one may choose an arbitrary $s_0 > 0$, for example $s_0 = 1$. In numerical calculations, it is important to avoid the removable singularity at $s = 0$, which is taken into account in our algorithm. Once $s = r - q$ has been found with sufficient accuracy, the remaining constants are then calculated as follows:

$$r = \frac{1}{2} \ln \left[\exp(s) \times \frac{L - (y_1 - y_2)}{L + (y_1 - y_2)} \right] \quad (23)$$

$$q = r - s \quad (24)$$

$$a = \frac{x_2 - x_1}{s} \quad (25)$$

$$b = x_1 - aq \quad (26)$$

$$\lambda = \frac{a \rho g}{2} \left[\exp\left(\frac{x_1 - b}{a}\right) + \exp\left(-\frac{x_1 - b}{a}\right) \right] - y_1 \rho g \quad (27)$$

Let us calculate the tension force of the curve, $\vec{T} = \begin{pmatrix} T_x \\ T_y \end{pmatrix}$, which must be directed tangentially to the curve. This condition can be expressed analytically as the equality to zero of the scalar product between $\vec{T}$ and the unit normal field $\vec{n} = \begin{pmatrix} n_x \\ n_y \end{pmatrix} = \frac{1}{\sqrt{1+\dot{y}^2(x)}} \begin{pmatrix} \dot{y}(x) \\ -1 \end{pmatrix}$ to the curve:

$$(\vec{T}, \vec{n}) = \frac{1}{\sqrt{1+\dot{y}^2(x)}} (T_x \dot{y}(x) - T_y) \equiv 0 \quad (28)$$

The projection T_x remains constant along the entire catenary curve, since the force of gravity acting on each infinitesimal element is directed only along the y -axis and has no horizontal component. Therefore, from the equilibrium condition for the element along the x -axis, it follows that the horizontal projections of the tension forces at its ends mutually compensate each other: $T_x(x+dx) - T_x(x) = 0$, hence $T_x = \text{const}$. In the vertical direction, the situation is different: the weight of the element is compensated by the difference between the vertical components of the tension at its ends. Thus, from (28) we obtain:

$$T_y(x) = T_x \dot{y}(x) \quad (29)$$

The force T_x can be calculated from the condition of vertical equilibrium of the entire catenary curve:

$$T_y(x_2) - T_y(x_1) = T_x \dot{y}(x_2) - T_x \dot{y}(x_1) = g\rho L \quad (30)$$

$$T_x = \frac{g\rho L}{\dot{y}(x_2) - \dot{y}(x_1)} \quad (31)$$

where ρL is the mass of the whole curve, $L = \frac{a}{2} \left[\exp\left(\frac{x_2-b}{a}\right) - \exp\left(-\frac{x_2-b}{a}\right) - \exp\left(\frac{x_1-b}{a}\right) + \exp\left(-\frac{x_1-b}{a}\right) \right]$ is the curve length (see (11)). Calculating $\dot{y}(x) = \frac{1}{2} \left[\exp\left(\frac{x-b}{a}\right) - \exp\left(-\frac{x-b}{a}\right) \right]$, we obtain from (31) and (29):

$$T_x = g\rho a \quad (32)$$

$$T_y(x) = \frac{g\rho a}{2} \left[\exp\left(\frac{x-b}{a}\right) - \exp\left(-\frac{x-b}{a}\right) \right] \quad (33)$$

For the magnitude of the total tangential tension force, we obtain:

$$|\vec{T}(x)| = T(x) = \sqrt{T_x^2 + T_y^2} = g\rho a \sqrt{1 + \frac{1}{4} \left[\exp\left(\frac{x-b}{a}\right) - \exp\left(-\frac{x-b}{a}\right) \right]^2} \quad (34)$$

If the value of b calculated in (26) satisfies $b \in (x_1, x_2)$, then $T_y(b) = 0$ and $T(b) = T_{\min}$. If $b \notin (x_1, x_2)$, then $T_{\min} = \min \{T(x_1), T(x_2)\}$. In any case, $T_{\max} = \max \{T(x_1), T(x_2)\}$.

For applications, it is useful to calculate the tensile stress in pascals:

$$\sigma(x) = \frac{T(x)}{S} = \frac{g\rho a}{S} \sqrt{1 + \frac{1}{4} \left[\exp\left(\frac{x-b}{a}\right) - \exp\left(-\frac{x-b}{a}\right) \right]^2} \quad (35)$$

where S is the constant cross-sectional area of the curve (m^2). If the value of b calculated in (26) satisfies $b \in (x_1, x_2)$, then $\sigma(b) = \sigma_{\min}$. If $b \notin (x_1, x_2)$, then $\sigma_{\min} = \min \{\sigma(x_1), \sigma(x_2)\}$. In any case, $\sigma_{\max} = \max \{\sigma(x_1), \sigma(x_2)\}$.

It is precisely $\sigma_{\max}$, before the onset of noticeable plastic deformation of the material, that is the critical parameter when choosing the cross-section and length of the conductor:

Conductor material	Material condition	Yield strength, Pa
Copper	soft, annealed	$3.0 \times 10^7 - 7.0 \times 10^7$ Pa
Copper	hard-drawn	$2.5 \times 10^8 - 3.6 \times 10^8$ Pa
Electrical-grade aluminium	soft, annealed	$2.0 \times 10^7 - 3.0 \times 10^7$ Pa
Electrical-grade aluminium	work-hardened, hard-drawn	$7.0 \times 10^7 - 1.65 \times 10^8$ Pa
Low-carbon steel	ordinary mild steel	$1.7 \times 10^8 - 2.8 \times 10^8$ Pa
Low-carbon steel	cold-drawn	$3.5 \times 10^8 - 5.0 \times 10^8$ Pa
High-strength steel wire	high-carbon / spring steel	$1.0 \times 10^9 - 1.8 \times 10^9$ Pa

For engineering practice, it is better to use not the yield strength itself, σ_{yield} , but the allowable tensile stress σ_{allow} : $\sigma_{\text{allow}} = \frac{\sigma_{\text{yield}}}{n}$, where n is the safety factor. For example, for $n = 3$.

Examples of the application of catenary curves in architecture:

Catenary arches under the roof of Gaudí's Casa Milà, Barcelona, Spain.



Antonio Gaudí's catenary model in Casa Milà.



Sheffield Winter Garden is surrounded by a series of catenary-shaped arches.

Example of calculating the catenary curve and the tensile stresses using a Python program, the code of which is given at the end of the report:

vd = 2710 # wire volume density (kg/m³)

S = 1.5 # wire cross-section (mm²)

p1 = (0.0, 20.0) # coordinates (m, m) of the left fixed end

p2 = (100.0, 30.0) # coordinates (m, m) of the right fixed end

Minimum wire length Lmin = 100.499 m

Choose a wire length (m), larger than Lmin: 110

Parameters of the catenary curve:

a = 66.99779571324274

b = 43.89242879189913

lambda = 2.4683119255540493

rho = 0.004065 kg / m

Tension in the catenary curve:

T(x1) = 3.2658649255540495 N

T(x2) = 3.664641425554049 N

Tmin = 2.6717146482241945 N

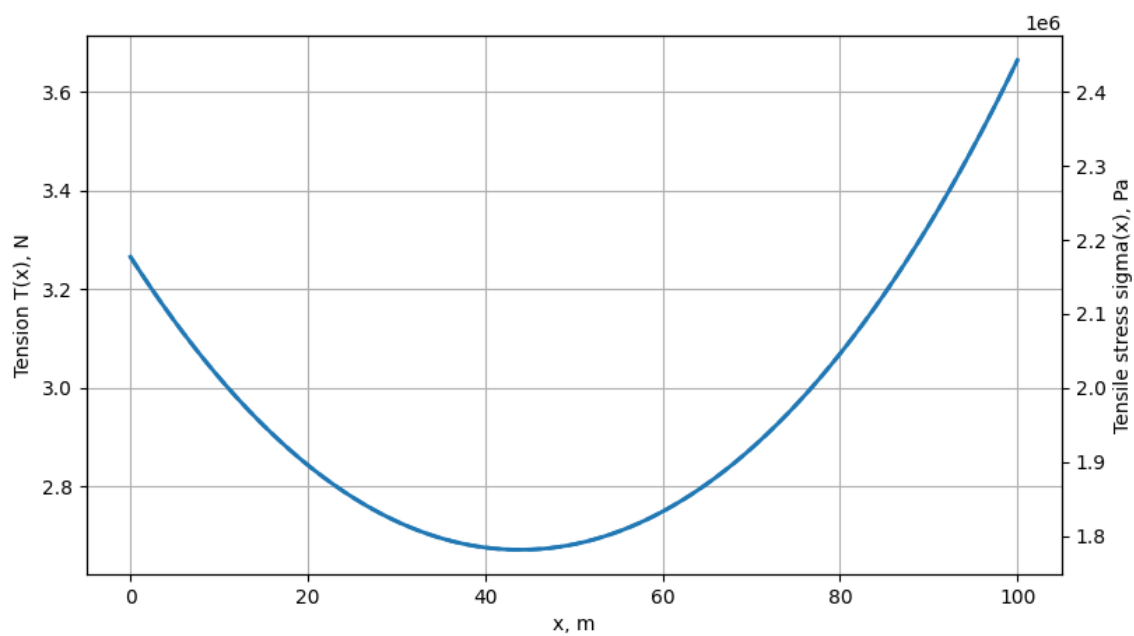
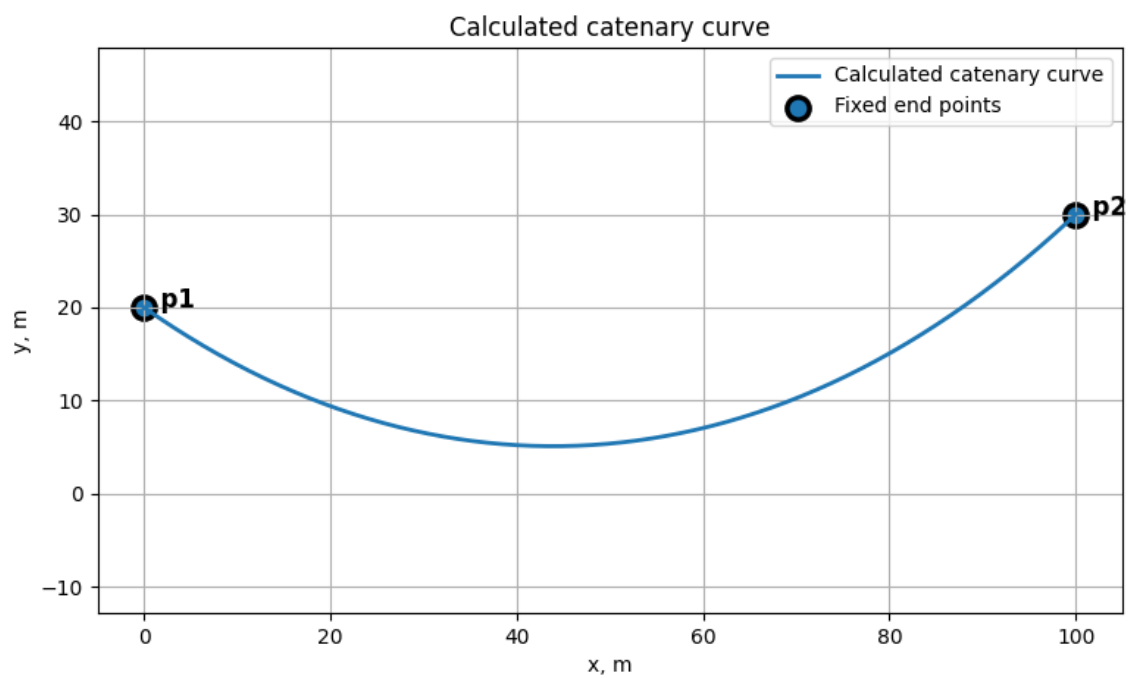
Tmax = 3.664641425554049 N

sigma(x1) = 2.177243e+06 Pa

sigma(x2) = 2.443094e+06 Pa

sigma_min = 1.781143e+06 Pa

sigma_max = 2.443094e+06 Pa



Program code in Python:

```

#
# Calculation of the shape of a catenary curve and the tensile stresses developed in
it.
#
# Dmitriy Makhnovskiy, 24.06.2026
#

import numpy as np
import sys
import matplotlib.pyplot as plt

# Enter the initial parameters
vd = 2710 # wire volume density (kg/m^3)
S = 1.5 # wire cross-section (mm^2)
p1 = (0.0, 20.0) # coordinates (m, m) of the left fixed end
p2 = (100.0, 30.0) # coordinates (m, m) of the right fixed end

# Newton iteration parameters (recommended)
s0 = 1.0
tol=1e-12
max_iter=100

# Interactive length and recalculated S (do not change)
g = 9.81 # free fall acceleration (m / s^2)
S = S * 1.0e-6 # wire cross-section (m^2)
x1 = p1[0]
y1 = p1[1]
x2 = p2[0]
y2 = p2[1]

if x2 <= x1:
    print("Error: the coordinate x2 must be larger than x1.")
    print("Program stopped.")
    sys.exit()

Lmin = np.sqrt((x2 - x1)**2 + (y2 - y1)**2) # minimum wire length

while True:
    print(f"Minimum wire length Lmin = {Lmin:.6g} m")
    try:
        L = float(input("Choose a wire length (m), larger than Lmin: "))
    except ValueError:
        print("Invalid input. Please enter a number.")
        continue
    if L > Lmin:
        break
    print("Wire length must be larger than Lmin.")

rho = vd * S # linear mass density (kg/m)

def f(s):
    C = np.sqrt(L**2 - (y1 - y2)**2) / (x2 - x1)
    if abs(s) < 1e-8:
        # limit of (exp(s)-1)/(s*exp(s/2)) as s -> 0
        phi = 1 + s**2 / 24
    else:
        phi = (np.exp(s) - 1) / (s * np.exp(s / 2))
    return phi - C

def df(s):
    if abs(s) < 1e-8:

```

```

    # derivative near s = 0
    return s / 12
else:
    numerator = (
        (s / 2) * np.exp(3 * s / 2)
        - np.exp(3 * s / 2)
        + np.exp(s / 2)
        + (s / 2) * np.exp(s / 2)
    )
    denominator = s**2 * np.exp(s)
    return numerator / denominator

def newton_s():
    s = s0
    for _ in range(max_iter):
        fs = f(s)
        dfs = df(s)
        s_new = s - fs / dfs
        error = abs(s_new - s)
        if error < tol:
            return s_new
        s = s_new
    raise RuntimeError("Newton method did not converge")

s = newton_s()
dy = y1 - y2
r = 0.5 * np.log(np.exp(s) * (L - dy) / (L + dy))
q = r - s
a = (x2 - x1) / s
b = x1 - a * q
lam = (a * rho * g / 2) * (np.exp((x1 - b) / a) + np.exp(-(x1 - b) / a)) - y1 * rho * g

print('')
print('Parameters of the catenary curve:')
print('a = ', a)
print('b = ', b)
print('lambda = ', lam)
print('rho = ', rho, ' kg / m')

print('')
print('Tension in the catenary curve:')
u1 = (x1 - b) / a
u2 = (x2 - b) / a

T1 = g * rho * a * np.sqrt(1 + 0.25 * (np.exp(u1) - np.exp(-u1))**2)
T2 = g * rho * a * np.sqrt(1 + 0.25 * (np.exp(u2) - np.exp(-u2))**2)
sigma1 = (g * rho * a / S) * np.sqrt(1 + 0.25 * (np.exp(u1) - np.exp(-u1))**2)
sigma2 = (g * rho * a / S) * np.sqrt(1 + 0.25 * (np.exp(u2) - np.exp(-u2))**2)

if x1 <= b <= x2:
    Tmin = g * rho * a
    sigma_min = g * rho * a / S
else:
    Tmin = min(T1, T2)
    sigma_min = min(sigma1, sigma2)

Tmax = max(T1, T2)
sigma_max = max(sigma1, sigma2)

print("T(x1) =", T1, "N")
print("T(x2) =", T2, "N")
print("Tmin =", Tmin, "N")

```

```
print("Tmax =", Tmax, "N")
print('')
print(f"sigma(x1) = {sigma1:.6e} Pa")
print(f"sigma(x2) = {sigma2:.6e} Pa")
print(f"sigma_min = {sigma_min:.6e} Pa")
print(f"sigma_max = {sigma_max:.6e} Pa")

# Plotting the calculated catenary curve and checking the fixed end points
# Coordinate array
x = np.linspace(x1, x2, 500)
u = (x - b) / a

# Catenary curve y(x)
y = (a / 2.0) * (np.exp(u) + np.exp(-u)) - lam / (rho * g)

# Tension T(x)
T = g * rho * a * np.sqrt(
    1.0 + 0.25 * (np.exp(u) - np.exp(-u))**2
)

# Tensile stress sigma(x)
sigma = T / S

# First figure: catenary curve and fixed end points
plt.figure(figsize=(9, 5))

plt.plot(x, y, linewidth=2, label="Calculated catenary curve")
plt.scatter([x1, x2], [y1, y2], s=120, linewidths=2.5, edgecolors="black", label="Fixed
end points")

plt.text(x1, y1, " p1", fontsize=12, fontweight="bold")
plt.text(x2, y2, " p2", fontsize=12, fontweight="bold")

plt.xlabel("x, m")
plt.ylabel("y, m")
plt.title("Calculated catenary curve")
plt.grid(True)
plt.axis("equal")
plt.legend()
plt.show()

# Second figure: T(x) and sigma(x)
fig, ax1 = plt.subplots(figsize=(9, 5))

ax1.plot(x, T, linewidth=2)
ax1.set_xlabel("x, m")
ax1.set_ylabel("Tension T(x), N")
ax1.grid(True)

ax2 = ax1.twinx()
ax2.plot(x, sigma, linewidth=2, linestyle="--")
ax2.set_ylabel("Tensile stress sigma(x), Pa")
ax2.ticklabel_format(axis="y", style="sci", scilimits=(0, 0))

plt.show()
```